-LAPORAN PEMROGRAMAN BERBASIS OBJECT

JOBSHEET 7



Nama:Daffa Dwi Nugroho

NIM : 254107020140

Kelas: TI 2-A

Link Github: https://github.com/Dapa-yap/Praktikum-Berbasis-Objek.git

D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

TAHUN AJARAN 2026

JOBSHEET 7

1. Praktikum
3.1 Percobaan 1
3.2 Karyawan

```java
public class Karyawan {
    private String nama;
    private String nip;
    private String golongan;
    private double gaji;

    public void setNama(String nama) {
        this.nama = nama;
    }
    public void setNip(String nip) {
        this.nip = nip;
    }
    public void setGolongan(String golongan) {
        this.golongan = golongan;
        switch (golongan) {
            case "1":
                this.gaji = 5000000;
                break;
            case "2":
                this.gaji = 3000000;
                break;
            case "3":
                this.gaji = 2000000;
                break;
            case "4":
                this.gaji = 1000000;
                break;
            case "5":
                this.gaji = 750000;
                break;
        }
    }
    public void setGaji(double gaji) {
        this.gaji = gaji;
    }
    public String getNama() {
        return nama;
    }
    public String getNip() {
        return nip;
    }
    public String getGolongan() {
        return golongan;
    }
    public double getGaji() {
        return gaji;
    }
}
```

3.3 Staff

```java
public class Staff extends Karyawan {
    private int lembur;
    private double gajiLembur;

    public void setLembur(int lembur){
        this.lembur = lembur;
    }
    public int getLembur(){
        return lembur;
    }
    public void setGajiLembur(double gajiLembur){
        this.gajiLembur = gajiLembur;
    }
    public double getGajiLembur(){
        return gajiLembur;
    }
    public double getGaji(int lembur, double gajiLembur){
        return super.getGaji() + (lembur * gajiLembur);
    }
    public double getGaji(){
        return super.getGaji() + (lembur * gajiLembur);
    }
    public void lihatInfo(){
        System.out.println("NIP: " + this.getNip());
        System.out.println("Nama: " + this.getNama());
        System.out.println("Golongan: " + this.getGolongan());
        System.out.println("Jml Lembur: " + this.getLembur());
        System.out.println("Gaji Lembur: " + this.getGajiLembur());
        System.out.println("Gaji: " + this.getGaji());
    }
}
```

3.4 Manager

```java
public class Manager extends Karyawan{
    private double tunjangan;
    private String bagian;
    private Staff st[];

    public void setTunjangan(double tunjangan){
        this.tunjangan = tunjangan;
    }
    public double getTunjangan(){
        return tunjangan;
    }
    public void setBagian(String bagian){
        this.bagian = bagian;
    }
    public String getBagian(){
        return bagian;
    }
    public void setStaff(Staff st[]){
        this.st = st;
    }
    public void viewStaff(){
        int i;
        System.out.println("-----------------------------------------------");
        for(i=0; i<st.length; i++){
            st[i].lihatInfo();
        }
            System.out.println("-----------------------------------------------");
        }
        public void lihatInfo(){
            System.out.println("Manager :" + this.getBagian());
            System.out.println("NIP: " + this.getNip());
            System.out.println("Nama: " + this.getNama());
            System.out.println("Golongan: " + this.getGolongan());
            System.out.println("Tunjangan: " + this.getTunjangan());
            System.out.println("Gaji: " + this.getGaji());
            System.out.println("Bagian: " + this.getBagian());
            this.viewStaff();
        }
        public double getGaji(){
            return super.getGaji() + tunjangan;
        }
    }
```

3.5 Utama

```java
public class Utama {
    public static void main (String [] args){
        System.out.println("Program Testing Class Manager & Staff");
        Manager man[] = new Manager[2];
        Staff staff1[] = new Staff[2];
        Staff staff2[] = new Staff[3];

        man[0]= new Manager();
        man[0].setNama("Tedjo");
        man[0].setNip("101");
        man[0].setGolongan("1");
        man[0].setTunjangan(5000000);
        man[0].setBagian("Administrasi");

        man[1]= new Manager();
        man[1].setNama("Atika");
        man[1].setNip("102");
        man[1].setGolongan("1");
        man[1].setTunjangan(2500000);
        man[1].setBagian("Pemasaran");

        staff1[0] = new Staff();
        staff1[0].setNama("Usman");
        staff1[0].setNip("0003");
        staff1[0].setGolongan("2");
        staff1[0].setLembur(10);
        staff1[0].setGajiLembur(10000);

        staff1[1] = new Staff();
        staff1[1].setNama("Anugrah");
        staff1[1].setNip("0005");
        staff1[1].setGolongan("2");
        staff1[1].setLembur(10);
        staff1[1].setGajiLembur(55000);
        man[0].setStaff(staff1);

        staff2[0] = new Staff();
        staff2[0].setNama("Hendra");
        staff2[0].setNip("0004");
        staff2[0].setGolongan("3");
        staff2[0].setLembur(15);
        staff2[0].setGajiLembur(55000);

        staff2[1] = new Staff();
        staff2[1].setNama("Arie");
        staff2[1].setNip("0006");
        staff2[1].setGolongan("4");
        staff2[1].setLembur(5);
        staff2[1].setGajiLembur(100000);

        staff2[2] = new Staff();
        staff2[2].setNama("Mentari");
        staff2[2].setNip("0007");
        staff2[2].setGolongan("3");
        staff2[2].setLembur(6);
        staff2[2].setGajiLembur(20000);
        man[1].setStaff(staff2);

        man[0].lihatInfo();
        man[1].lihatInfo();

    }
}
```

```
Program Testing Class Manager & Staff
Manager :Administrasi
NIP: 101
Nama: Tedjo
Golongan: 1
Tunjangan: 5000000.0
Gaji: 1.0E7
Bagian: Administrasi
-------------------------------------------------
NIP: 0003
Nama: Usman
Golongan: 2
Jml Lembur: 10
Gaji Lembur: 10000.0
Gaji: 3100000.0
NIP: 0005
Nama: Anugrah
Golongan: 2
Jml Lembur: 10
Gaji Lembur: 55000.0
Gaji: 3550000.0
-------------------------------------------------
Manager :Pemasaran
NIP: 102
Nama: Atika
Golongan: 1
Tunjangan: 2500000.0
Gaji: 7500000.0
Bagian: Pemasaran
-------------------------------------------------
NIP: 0004
Nama: Hendra
```

```
NIP: 0004
Nama: Hendra
Golongan: 3
Jml Lembur: 15
Gaji Lembur: 55000.0
Gaji: 2825000.0
NIP: 0006
Nama: Arie
Golongan: 4
Jml Lembur: 5
Gaji Lembur: 100000.0
Gaji: 1500000.0
NIP: 0007
Nama: Mentari
Golongan: 3
Jml Lembur: 6
Gaji Lembur: 20000.0
Gaji: 2120000.0
```

4. Latihan

```java
public class PerkalianKu {
    void perkalian(int a, int b){
        System.out.println(a*b);
    }
    void perkalian(int a, int b, int c){
        System.out.println(a*b*c);
    }
    Run | Debug
    public static void main(String[] args) {
        PerkalianKu obj = new PerkalianKu();
        obj.perkalian(a: a: 25, b: b: 43);
        obj.perkalian(a: a: 34, b: b: 23, c: c: 56);
    }
}
```

```
1075
43792
```

4.1 Dari source coding diatas terletak dimanakah overloading?

Overloading terletak pada method perkalian di dalam class PerkalianKu. Ada dua method dengan nama sama (perkalian) tetapi memiliki daftar parameter yang berbeda:

```
void perkalian(int a, int b){
    System.out.println(a*b);
}
void perkalian(int a, int b, int c){
    System.out.println(a*b*c);
}
```

4.2 Jika terdapat overloading ada berapa jumlah parameter yang berbeda?

Ada 2 variasi jumlah parameter yang berbeda, yaitu:

- Method perkalian dengan 2 parameter (int a, int b)
- Method perkalian dengan 3 parameter (int a, int b, int c)

```
public class PerkalianKu {
    void perkalian(int a, int b){
        System.out.println(a*b);
    }
    void perkalian(double a, double b){
        System.out.println(a*b);
    }
    Run | Debug
    public static void main(String[] args) {
        PerkalianKu obj = new PerkalianKu();
        obj.perkalian(a: a: 25, b: b: 43);
        obj.perkalian(a: a: 34.56, b: b: 23.7);
    }
}
```

4.3 Dari source coding diatas terletak dimanakah overloading?

Overloading terletak pada method perkalian di dalam class PerkalianKu. Ada dua method dengan nama sama (perkalian) dan jumlah parameter sama (2), tetapi tipe datanya berbeda

```
void perkalian(int a, int b){
    System.out.println(a*b);
}
void perkalian(double a, double b){
    System.out.println(a*b);
}
```

4.4Jika terdapat overloading ada berapa tipe parameter yang berbeda?

Ada 2 tipe parameter yang berbeda, yaitu:

1. int pada method perkalian(int a, int b)
2. double pada method perkalian(double a, double b)

```
class Ikan {
    public void swim(){
        System.out.println(x: x: "Ikan bisa berenang");
    }
}
class Piranha extends Ikan{
    public void swim(){
        System.out.println(x: x: "Piranha bisamakan daging")
    }
}
public class Fish {
    Run | Debug
    public static void main(String[] args) {
        Ikan a = new Ikan();
        Ikan b = new Piranha();
        a.swim();
        b.swim();
    }
}
```

4.5 Dari source coding diatas terletak dimanakah overriding?

Overriding terletak pada method swim(). Class Ikan (superclass) punya method swim(), lalu class Piranha (subclass, extends Ikan) menulis ulang method swim() dengan nama, parameter, dan tipe kembalian yang sama, tetapi isinya berbeda

```
class Ikan {
    public void swim(){
        System.out.println(x: x: "Ikan bisa berenang");
    }
}
class Piranha extends Ikan{
    public void swim(){
        System.out.println(x: x: "Piranha bisamakan daging");
    }
}
```

4.6 Jabarkanlah apabila sour coding diatas jika terdapat overriding?

Penjabarannya:

1. Class Ikan adalah superclass yang memiliki method swim() dengan output Ikan bisa berenang.
2. Class Piranha adalah subclass yang mewarisi Ikan (extends). Piranha mendefinisikan ulang swim() dengan perilaku sendiri, yaitu Piranha bisa makan daging. Inilah yang disebut overriding.
3. Pada method main (class Fish), dibuat dua objek:
    - Ikan a = new Ikan(); objek bertipe Ikan, sehingga a.swim() menjalankan method milik Ikan.
    - Ikan b = new Piranha(); variabel bertipe Ikan tetapi objeknya adalah Piranha. Karena swim() di-override, b.swim() menjalankan method milik Piranha. Java menentukan hal ini saat program berjalan (runtime polymorphism atau dynamic method dispatch).

```
Ikan bisa berenang
Piranha bisa makan daging
PS C:\Users\Daffa Dwi\OneDri
```

5. Tugas

5.1 Overloading

Implementasikan konsep overloading pada class diagram dibawah ini :

```java
public class Segitiga {
    private int sudut;

    public int totalSudut(int sudutA){
        sudut = 180 - sudutA;
        return sudut;
    }
    public int totalSudut(int sudutA, int sudutB){
        sudut = 180 - (sudutA + sudutB);
        return sudut;
    }
    public int keliling(int sisiA, int sisiB, int sisiC){
        return sisiA + sisiB + sisiC;
    }
    public double keliling(int sisiA, int sisiB){
        double c = Math.sqrt(Math.pow(sisiA, 2) + Math.pow(sisiB, 2));
        return sisiA + sisiB + c;
    }
        public static void main(String[] args) {
        Segitiga s = new Segitiga();

        System.out.println("Sudut ketiga (1 sudut diketahui)  : " + s.totalSudut(90));
        System.out.println("Sudut ketiga (2 sudut diketahui)  : " + s.totalSudut(60, 50));
        System.out.println("Keliling (3 sisi)                 : " + s.keliling(3, 4, 5));
        System.out.println("Keliling (2 sisi, siku-siku)      : " + s.keliling(3, 4));
    }

}
```

```
Sudut ketiga (1 sudut diketahui)  : 90
Sudut ketiga (2 sudut diketahui)  : 70
Keliling (3 sisi)                 : 12
Keliling (2 sisi, siku-siku)      : 12.0
PS C:\Users\Daffa Dwi\OneDrive\Documents\S
```

5.2 Overriding

Implementasikan class diagram dibawah ini dengan menggunakan teknik dynamic method dispatch :

```java
class Manusia {
    public void bernafas(){
        System.out.println("Manusia bisa bernafas");
    }
    public void makan(){
        System.out.println("Manusia bisa makan");
    }
}
class Dosen extends Manusia{
    public void makan(){
        System.out.println("Dosen bisa makan");
    }
    public void lembur(){
        System.out.println("Dosen bisa lembur");
    }
}
class Mahasiswa extends Manusia{
    public void makan(){
        System.out.println("Mahasiswa bisa makan");
    }
    public void tidur(){
        System.out.println("Mahasiswa bisa tidur");
    }
}
public class Overriding {
    public static void main(String[] args) {
        Manusia m;
        m = new Manusia();
        m.bernafas();
        m.makan();
        System.out.println();
        m = new Dosen();
        m.bernafas();
        m.makan();
        ((Dosen) m).lembur();
        System.out.println();
        m = new Mahasiswa();
        m.bernafas();
        m.makan();
        ((Mahasiswa) m).tidur();
    }
}

```

```
Manusia bisa bernafas
Manusia bisa makan

Manusia bisa bernafas
Dosen bisa makan
Dosen bisa lembur

Manusia bisa bernafas
Mahasiswa bisa makan
Mahasiswa bisa tidur
```